

UNIVERSIDAD TECNOLÓGICA

Ingeniería en Desarrollo y Gestión de Software

Materia: DevOps

Proyecto Integrador:

**Pipeline CI/CD automatizado para una API REST con
Docker, GitHub Actions y AWS EC2**

Integrantes:

Diego Garcia Martínez

Grupo: IDGS16

Profesor: Emmanuel Martínez Hernández

5 de octubre de 2026

Índice

1. Introducción	3
2. Objetivos	4
2.1. Objetivo general	4
2.2. Objetivos específicos	4
2.3. Entregables	4
3. Herramientas utilizadas	5
4. Arquitectura de la solución	5
5. Resultados	6
5.1. Desarrollo de la API REST	6
5.2. Pruebas automatizadas y cobertura	8
5.3. Contenedorización	9
5.4. Integración continua con GitHub Actions	10
5.5. Gestión de artefactos en Docker Hub	12
5.6. Infraestructura en AWS EC2	12
5.7. Seguridad: GitHub Secrets	13
5.8. Despliegue continuo en EC2	14
5.9. Demostración práctica	15
6. Conclusiones	16

Índice de figuras

1.	Arquitectura del pipeline CI/CD implementado	6
2.	Estructura de archivos del repositorio	7
3.	Prueba de los 6 endpoints contra la IP pública de la EC2	8
4.	Ejecución local de las 31 pruebas con pytest	8
5.	Reporte de cobertura de código (umbral mínimo: 70 %)	9
6.	Dockerfile multi-etapa de la API	10
7.	Construcción y ejecución local de la imagen Docker	10
8.	Ejecución exitosa del pipeline con los jobs test, docker y deploy	11
9.	Resumen de cobertura mostrado en los logs de GitHub Actions	11
10.	Repositorio en Docker Hub con las etiquetas <code>latest</code> y el hash del commit .	12
11.	Instancia EC2 en ejecución en la consola de AWS	13
12.	Reglas de entrada del Security Group (puertos 22 y 80)	13
13.	Secretos configurados en el repositorio (GitHub nunca muestra sus valores) .	14
14.	Log del job deploy en GitHub Actions	15
15.	Contenedor <code>api</code> en ejecución en la EC2 (<code>docker ps</code>)	15
16.	API respondiendo en el navegador en <code>http://18.223.115.56/api/health</code> .	15
17.	<code>/api/health</code> antes del cambio, y el <code>git commit</code> / <code>git push</code> de la demo . .	16
18.	<code>/api/health</code> después del <code>git push</code> : mensaje y commit nuevos	16

1. Introducción

El desarrollo de software moderno ya no termina cuando el programador escribe la última línea de código. Para que una aplicación genere valor tiene que llegar a producción de forma rápida, repetible y segura, y tiene que poder actualizarse muchas veces sin que los usuarios lo noten. Durante años el paso del código al servidor se hacía de forma manual: alguien compilaba el proyecto en su computadora, copiaba archivos al servidor, reiniciaba servicios y cruzaba los dedos. Este proceso era lento, dependía de la memoria de una persona y era una fuente constante de errores del tipo “en mi máquina sí funciona”. La cultura DevOps surge precisamente para cerrar la brecha entre quienes desarrollan y quienes operan el software, apoyándose en la automatización de todo el ciclo de vida de una aplicación [6].

Dos prácticas son el corazón de esa automatización. La **integración continua** (CI, *Continuous Integration*) consiste en que cada cambio que se sube al repositorio se compila y se prueba automáticamente, de modo que los errores se detectan minutos después de haberse introducido y no semanas después [3]. La **entrega y despliegue continuo** (CD, *Continuous Delivery/Deployment*) extiende esa idea: si las pruebas pasan, el mismo proceso empaqueta la aplicación y la publica en el servidor sin intervención humana [5]. Al conjunto ordenado de pasos que sigue el código desde el `git push` hasta producción se le llama *pipeline*.

Para que el pipeline sea confiable, el artefacto que se prueba debe ser exactamente el mismo que se ejecuta en producción. Aquí entran los **contenedores**: Docker permite empaquetar la aplicación junto con su intérprete y sus dependencias en una imagen inmutable que se comporta igual en la laptop del desarrollador, en el servidor de integración y en la nube [1]. Estas imágenes se guardan en un registro (en este proyecto, Docker Hub) y se identifican con etiquetas; usar el hash del commit como etiqueta permite saber con precisión qué versión del código está corriendo y regresar a una versión anterior si algo falla.

En este proyecto integrador se implementó un flujo de trabajo completo y profesional para el ciclo de vida de una API REST. Se desarrolló una API en Python con el microframework Flask [2] que expone 6 endpoints para la gestión de usuarios: las cinco operaciones CRUD (listar, consultar, crear, actualizar y eliminar) y un endpoint de salud que informa la versión desplegada. La API cuenta con 31 pruebas automatizadas escritas con pytest, que alcanzan una cobertura de código del 99 %, muy por encima del umbral mínimo exigido del 70 %. El pipeline, definido con GitHub Actions [4], ejecuta el análisis estático y las pruebas en cada *push* o *pull request* a la rama `main`; cuando el cambio llega a `main`, construye la imagen Docker, la publica en Docker Hub con las etiquetas `:latest` y `:<sha del commit>`, y finalmente se conecta por SSH a una instancia Ubuntu Server de Amazon EC2 [7] para descargar la versión

nueva, validarla, detener el contenedor antiguo y levantar el nuevo en el puerto 80.

Un aspecto central del proyecto es la seguridad: ninguna contraseña, token, dirección IP ni llave SSH aparece en el código del repositorio. Todos estos datos se administran como *GitHub Secrets*, y la autenticación con Docker Hub se realiza mediante un *Personal Access Token* en lugar de la contraseña de la cuenta. Además, el despliegue incluye una verificación de salud: la versión nueva se prueba en un puerto temporal antes de reemplazar a la que está en producción, de forma que una imagen defectuosa nunca deja al servicio fuera de línea.

El presente reporte documenta la arquitectura de la solución, el proceso seguido en cada etapa y los resultados obtenidos, apoyados con figuras numeradas que muestran la evidencia de cada paso, incluyendo la demostración en vivo en la que un cambio de una línea en la API recorre automáticamente todo el pipeline hasta quedar publicado en la nube.

2. Objetivos

2.1. Objetivo general

Implementar un pipeline CI/CD automatizado que pruebe, empaquete y despliegue una API REST en AWS EC2 cada vez que se publica un cambio en la rama `main` del repositorio.

2.2. Objetivos específicos

- Desarrollar una API REST con al menos 6 endpoints funcionales.
- Implementar pruebas unitarias y de integración con una cobertura mínima del 70 %.
- Contenedorizar la aplicación con un `Dockerfile` optimizado y un `.dockerignore`.
- Automatizar pruebas, construcción y publicación de la imagen con GitHub Actions y Docker Hub.
- Desplegar automáticamente en una instancia EC2 por SSH, gestionando los datos sensibles con GitHub Secrets.

2.3. Entregables

- **Repositorio de GitHub:** <https://github.com/2023371031-del/sa-cicd-api>
- **URL pública de la API:** <http://18.223.115.56/api/health>

3. Herramientas utilizadas

Tabla 1: Tecnologías utilizadas en el proyecto

Herramienta	Uso en el proyecto
Python 3.12 + Flask 3.1	Lenguaje y framework de la API REST
Gunicorn 23	Servidor WSGI de producción dentro del contenedor
pytest + pytest-cov	Pruebas automatizadas y reporte de cobertura
flake8	Análisis estático (lint) del código
Docker	Construcción de la imagen y ejecución del contenedor
Docker Hub	Registro de imágenes (artefactos)
GitHub / GitHub Actions	Repositorio de código y motor del pipeline CI/CD
AWS EC2 (Ubuntu Server 26.04, t3.micro)	Servidor en la nube donde se ejecuta la API
Visual Studio Code, curl	Edición de código y pruebas manuales de endpoints

4. Arquitectura de la solución

La Figura 1 resume el flujo completo. El desarrollador hace `git push` a la rama `main`; GitHub Actions ejecuta tres *jobs* encadenados (`test`, `docker` y `deploy`), cada uno de los cuales solo inicia si el anterior terminó con éxito. El resultado final es un contenedor actualizado en la instancia EC2 que atiende peticiones HTTP en el puerto 80.

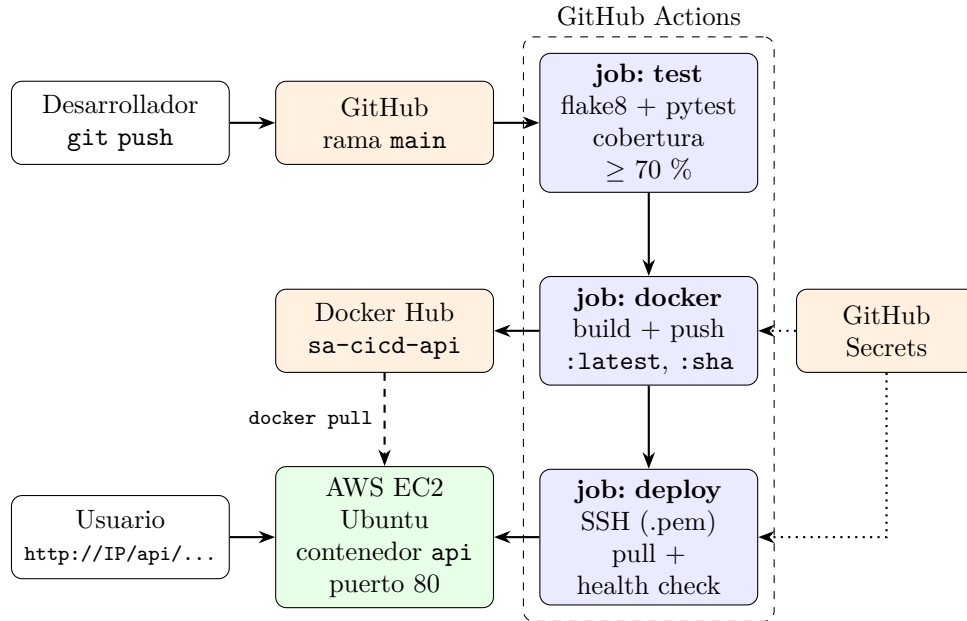


Figura 1: Arquitectura del pipeline CI/CD implementado

El código se organizó por capas, siguiendo la estructura vista en clase: **routes/** contiene las rutas (**users.py** y **system.py**), **services/** la lógica de negocio y las validaciones, **data/** el almacenamiento, y **tests/** las pruebas. En la raíz se encuentran el **Dockerfile**, el **.dockerignore** y el workflow en **.github/workflows/main.yml**.

5. Resultados

5.1. Desarrollo de la API REST

Se construyó una API de gestión de usuarios con **6 endpoints**: cinco operaciones CRUD sobre el recurso **users** y un endpoint de salud (**/api/health**) que además informa la versión, un mensaje configurable y el hash del commit desplegado. El código se separó en capas: las rutas (**routes/users.py**) solo reciben la petición y devuelven la respuesta; la lógica de negocio (**services/user_service.py**) valida los datos contra un esquema de campos, revisa que el email no se repita y modifica los datos; y **data/store.py** mantiene los usuarios en memoria a partir de **data/seed.json**. La Figura 2 muestra la organización del proyecto y la Tabla 2 resume los endpoints.

```

PowerShell — sa-cicd-api

$ tree sa-cicd-api
sa-cicd-api/
├── .github/
│   └── workflows/
│       └── main.yml
├── data/
│   ├── __init__.py
│   ├── seed.json
│   └── store.py
├── reporte/
│   ├── figuras/
│   │   └── ...
│   ├── reporte.pdf
│   └── reporte.tex
├── routes/
│   ├── __init__.py
│   ├── responses.py
│   ├── system.py
│   └── users.py
├── scripts/
│   └── setup_ec2.sh
├── services/
│   ├── __init__.py
│   ├── user_service.py
│   └── validators.py
├── tests/
│   ├── conftest.py
│   ├── test_system.py
│   └── test_users.py
├── .coveragerc
├── .dockerignore
├── .flake8
├── .gitattributes
├── .gitignore
├── app.py
├── config.py
├── Dockerfile
├── pytest.ini
├── README.md
├── requirements-dev.txt
└── requirements.txt

```

Figura 2: Estructura de archivos del repositorio

Tabla 2: Endpoints de la API

#	Método	Ruta y descripción	Códigos
1	GET	/api/health — estado, versión, mensaje y commit	200
2	GET	/api/users — listar usuarios	200
3	GET	/api/users/{id} — consultar un usuario	200, 404
4	POST	/api/users — crear un usuario	201, 400, 409, 415
5	PUT	/api/users/{id} — actualizar uno o varios campos	200, 400, 404, 409, 415
6	DELETE	/api/users/{id} — eliminar un usuario	200, 404

Todas las respuestas comparten el formato `{"statusCode": ..., "data": ...}` y los errores `{"statusCode": ..., "error": "mensaje"}`, con códigos HTTP coherentes: 201 al crear, 400 ante datos inválidos (campos faltantes, vacíos, de tipo incorrecto, email mal formado o campos no permitidos como `id`), 404 si el usuario no existe, 409 si el email ya está registrado y 415 cuando el cuerpo no es JSON. La Figura 3 muestra la prueba de los seis endpoints con `curl` directamente contra la IP pública de la EC2: se consulta el estado, se listan los usuarios, se consulta uno, se crea el usuario 4, se actualiza su rol y finalmente se elimina.

```
PowerShell — Prueba de los 6 endpoints contra la EC2

$ curl http://18.223.115.56/api/health
{"statusCode": 200, "data": {"status": "ok", "version": "1.0.0", "message": "Hola desde el pipeline CI/CD - IDGS16", "commit": "47737e5dfc45314500b24fcaedbc98a6c3a145a4"}}

$ curl http://18.223.115.56/api/users
{"statusCode": 200, "data": [{"id": 1, "name": "Ana López", "email": "ana@mail.com", "role": "admin", "active": true}, {"id": 2, "name": "Luis Pérez", "email": "luis@mail.com", "role": "user", "active": true}, {"id": 3, "name": "María García", "email": "maria@mail.com", "role": "user", "active": false}]}

$ curl http://18.223.115.56/api/users/1
{"statusCode": 200, "data": {"id": 1, "name": "Ana López", "email": "ana@mail.com", "role": "admin", "active": true}}

$ curl -X POST http://18.223.115.56/api/users -H "Content-Type: application/json" -d '{"name":"Pedro","email":"pedro@mail.com"}'
{"statusCode": 201, "data": {"id": 4, "name": "Pedro", "email": "pedro@mail.com", "role": "user", "active": true}}

$ curl -X PUT http://18.223.115.56/api/users/4 -H "Content-Type: application/json" -d '{"role":"admin"}'
{"statusCode": 200, "data": {"id": 4, "name": "Pedro", "email": "pedro@mail.com", "role": "admin", "active": true}}

$ curl -X DELETE http://18.223.115.56/api/users/4
{"statusCode": 200, "data": {"message": "Usuario eliminado", "id": 4}}
```

Figura 3: Prueba de los 6 endpoints contra la IP pública de la EC2

5.2. Pruebas automatizadas y cobertura

Las pruebas se escribieron con `pytest` usando el cliente de pruebas de `Flask`, que permite hacer peticiones HTTP a la aplicación sin levantar un servidor. Un *fixture* recarga los datos iniciales antes de cada prueba para que sean independientes entre sí. Se implementaron 31 pruebas en dos archivos: `test_users.py` prueba los cinco endpoints de usuarios, tanto los casos exitosos como los errores que puede provocar quien consume la API (email repetido o inválido, campos vacíos, tipos incorrectos, campos no permitidos, cuerpo sin JSON, ids inexistentes y método no permitido); para los casos de validación se usó *parametrización*, de modo que una sola función de prueba se ejecuta con nueve cuerpos inválidos distintos. Por su parte, `test_system.py` cubre el endpoint de salud y el manejo de errores 404 y 500. La Figura 4 muestra la ejecución.

```
$ pytest -v
tests/test_system.py::test_health PASSED [ 3%]
tests/test_system.py::test_route_not_found PASSED [ 6%]
tests/test_system.py::test_internal_error_handler PASSED [ 9%]
tests/test_users.py::test_list_users PASSED [ 12%]
tests/test_users.py::test_get_user PASSED [ 16%]
tests/test_users.py::test_get_user_not_found PASSED [ 19%]
...
tests/test_users.py::test_delete_user_not_found PASSED [ 96%]
tests/test_users.py::test_delete_all_not_allowed PASSED [100%]
===== 31 passed in 0.57s =====
```

Figura 4: Ejecución local de las 31 pruebas con `pytest`

La cobertura se mide con `pytest-cov`. En `pytest.ini` se configuró la opción `--cov-fail-under=70`, de modo que si la cobertura baja del 70 % el comando termi-

na con error y el pipeline se detiene. Como se observa en la Figura 5, se obtuvo una cobertura total del **99.48 %**: de 191 sentencias solo una no se ejecuta.

Name	Stmts	Miss	Cover	Missing

app.py	24	0	100%	
config.py	2	0	100%	
data/__init__.py	0	0	100%	
data/store.py	25	0	100%	
routes/__init__.py	3	0	100%	
routes/responses.py	13	0	100%	
routes/system.py	8	0	100%	
routes/users.py	20	0	100%	
services/__init__.py	0	0	100%	
services/user_service.py	46	0	100%	
services/validators.py	50	1	98%	33

TOTAL	191	1	99%	
Required test coverage of 70% reached. Total coverage: 99.48%				
31 passed in 0.57s				

Figura 5: Reporte de cobertura de código (umbral mínimo: 70 %)

5.3. Contenedorización

El Dockerfile (Figura 6) usa una construcción en dos etapas (*multi-stage build*): la primera instala las dependencias en un entorno virtual y la segunda copia únicamente ese entorno y el código a una imagen `python:3.12-slim` limpia, sin compiladores ni cachés de pip. Además, el contenedor se ejecuta con un usuario sin privilegios (`appuser`), define un `HEALTHCHECK` contra `/api/health` y recibe el hash del commit como argumento de construcción para exponerlo en `/api/health`. Las dependencias se copian antes que el código para aprovechar la caché de capas: si solo cambia el código, no se reinstalan las librerías [1].

```

FROM python:3.12-slim AS builder
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PATH="/opt/venv/bin:$PATH"
RUN useradd --create-home --uid 1000 appuser
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
COPY --chown=appuser:appuser . .
ARG GIT_SHA=local
ENV GIT_SHA=$GIT_SHA
USER appuser
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s CMD python -c "import urllib.request; \
    urllib.request.urlopen('http://127.0.0.1:8000/api/health', timeout=2)" || exit 1
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "1", "--threads", "4", "app:
    create_app()"]

```

Figura 6: Dockerfile multi-etapa de la API

El archivo `.dockerignore` excluye de la imagen todo lo que no se necesita en producción: el directorio `.git`, cachés de Python, entornos virtuales, las pruebas, reportes de cobertura, archivos `.env`, llaves `.pem`, logs, la documentación y el reporte. Gracias a esto, la imagen final ocupa 49.4MB comprimida. La Figura 7 muestra la construcción y prueba local del contenedor.

```

$ docker build -t sa-cicd-api --build-arg GIT_SHA=test123 .
=> naming to docker.io/library/sa-cicd-api:latest           done
$ docker run -d --name sa-test -p 8089:8000 sa-cicd-api
$ curl http://localhost:8089/api/health
{"statusCode":200,"data":{"status":"ok","version":"1.0.0",
"message":"Hola desde el pipeline CI/CD - IDGS16","commit":"test123"}}
$ docker images sa-cicd-api
IMAGE                ID                                DISK USAGE    CONTENT SIZE
sa-cicd-api:latest   639a0ff38f75                      205MB         49.4MB

```

Figura 7: Construcción y ejecución local de la imagen Docker

5.4. Integración continua con GitHub Actions

El workflow `.github/workflows/main.yml` se dispara con eventos `push` y `pull_request` sobre la rama `main`. El primer job, `test`, instala Python 3.12, ejecuta `flake8` y luego `pytest -v`, que imprime en el log la tabla de cobertura; adicionalmente, el resumen de cobertura se publica en formato Markdown en la página de la ejecución (`$GITHUB_STEP_SUMMARY`) y el

archivo `coverage.xml` se guarda como artefacto. En los *pull requests* solo corre este job; los de construcción y despliegue se ejecutan únicamente con `push` a `main`. La Figura 8 muestra una ejecución exitosa del pipeline y la Figura 9 el resumen de cobertura en los logs.

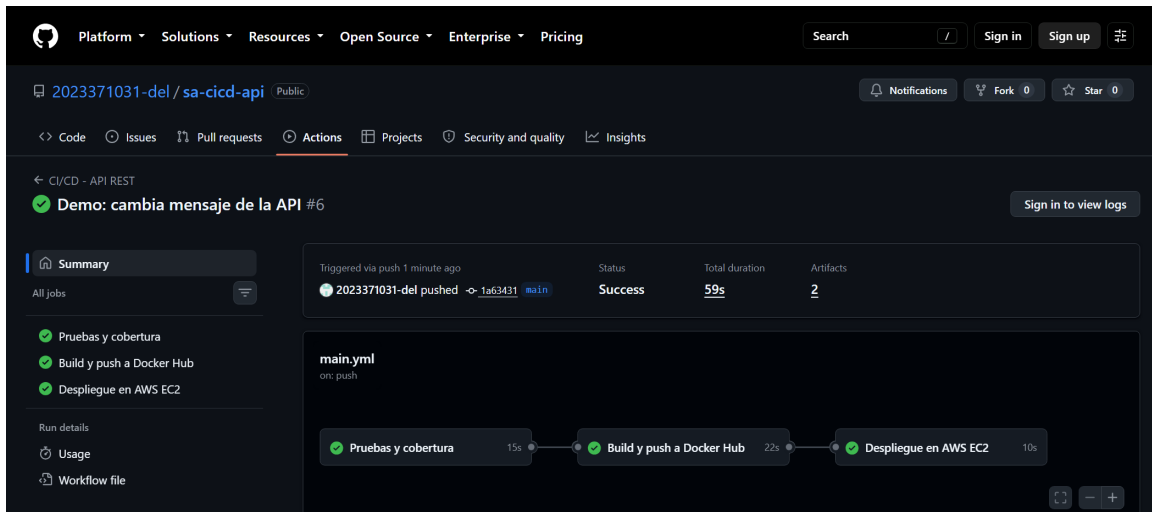


Figura 8: Ejecución exitosa del pipeline con los jobs test, docker y deploy

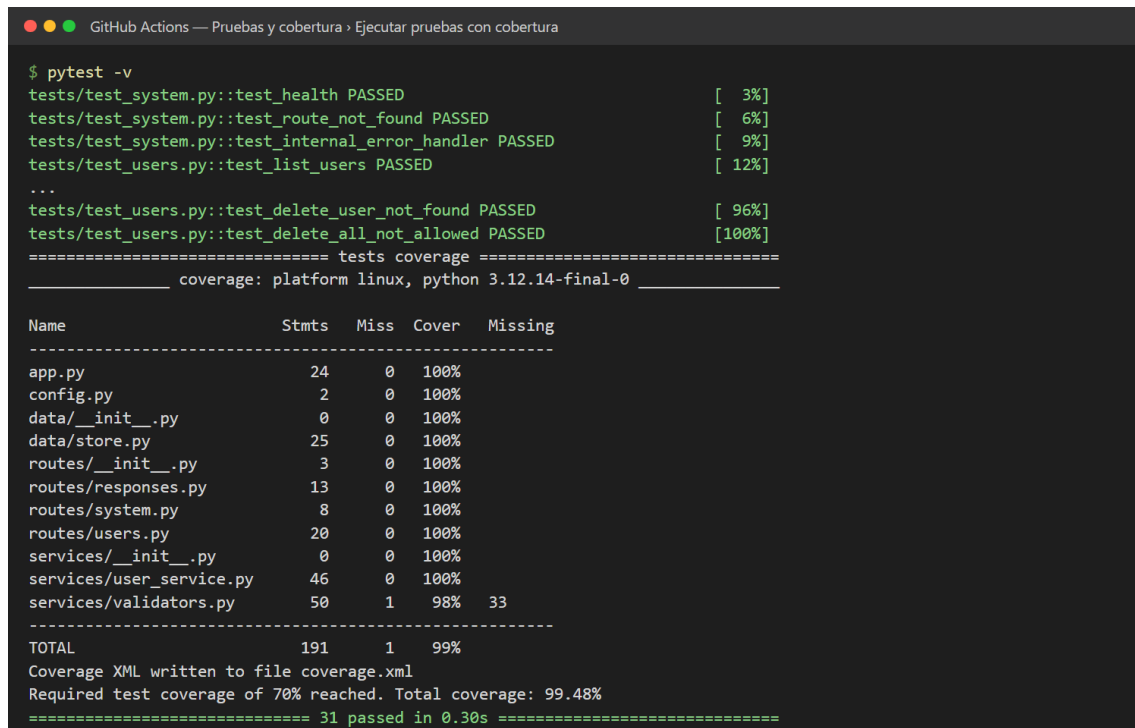


Figura 9: Resumen de cobertura mostrado en los logs de GitHub Actions

5.5. Gestión de artefactos en Docker Hub

El job `docker` se autentica en Docker Hub con la acción `docker/login-action` usando un *Personal Access Token* con permisos de lectura y escritura, almacenado en el secreto `DOCKERHUB_TOKEN`. Después construye la imagen con Buildx (con caché de capas entre ejecuciones) y la publica con dos etiquetas: `:latest`, que siempre apunta a la versión más reciente, y `:${{ github.sha }}`, que identifica de forma única el commit que la generó y permite regresar a cualquier versión anterior. La Figura 10 muestra el repositorio en Docker Hub con ambas etiquetas.

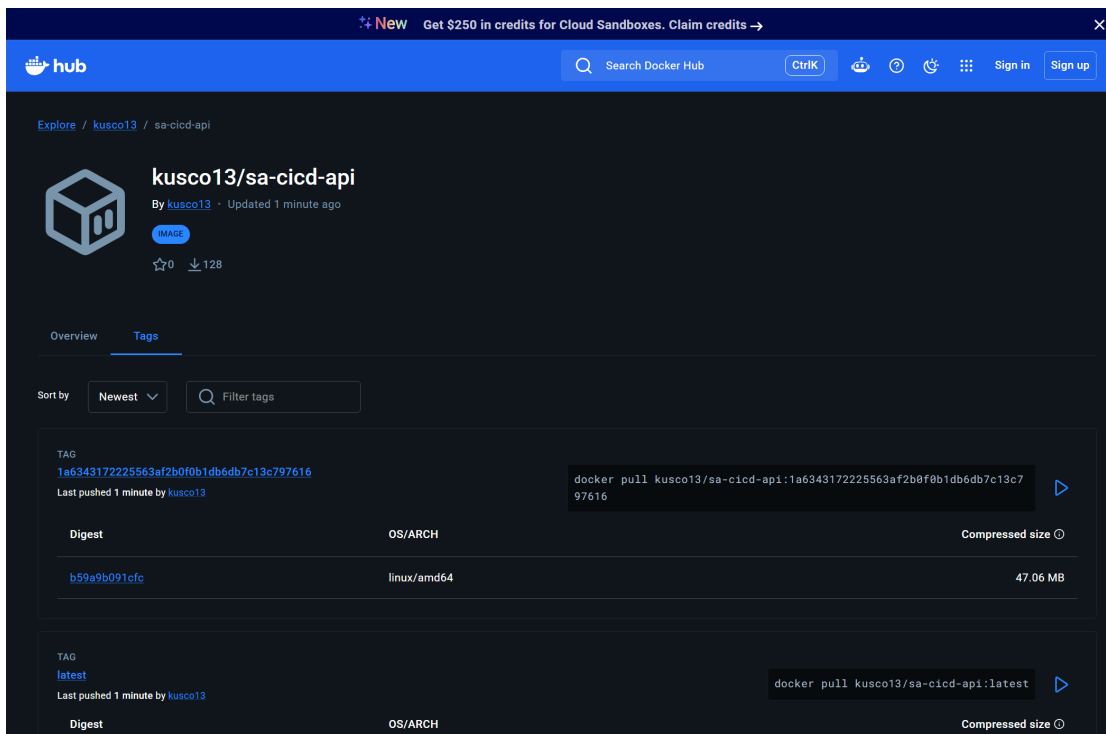


Figura 10: Repositorio en Docker Hub con las etiquetas `latest` y el hash del commit

5.6. Infraestructura en AWS EC2

Se utilizó una instancia EC2 `t3.micro` con Ubuntu Server 26.04 LTS en la región `us-east-2` (Ohio), con IP pública `18.223.115.56`, a la que se accede con un par de llaves en formato `.pem` (Figura 11). Se reutilizó la instancia de la práctica anterior para no consumir créditos adicionales; ya contaba con Docker Engine instalado y con el usuario `ubuntu` dentro del grupo `docker`. Para preparar una instancia nueva, el repositorio incluye el script `scripts/setup_ec2.sh`, que instala Docker y configura ese grupo. Al *Security Group* se le agregó la regla de entrada HTTP en el puerto 80, por donde se atiende a los usuarios,

y se abrió el puerto 22 (SSH) a cualquier origen, ya que GitHub Actions se conecta desde direcciones IP que cambian en cada ejecución (Figura 12).

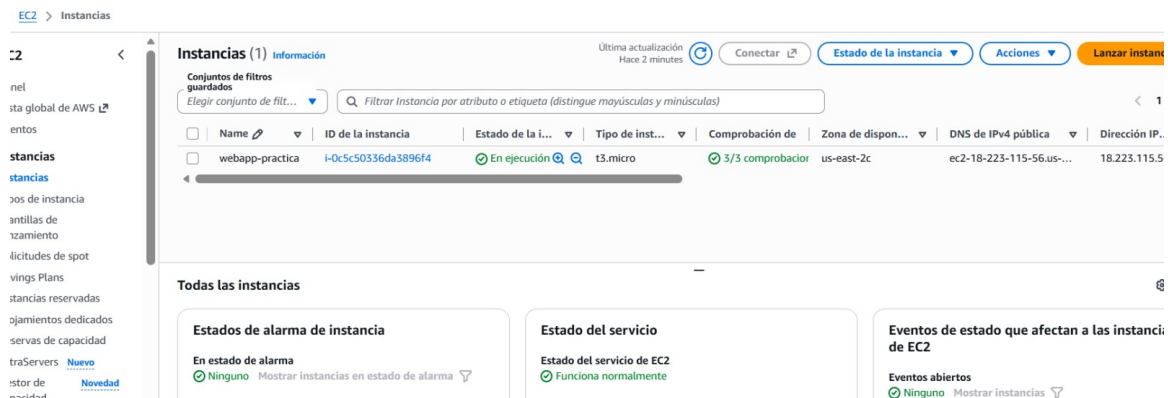


Figura 11: Instancia EC2 en ejecución en la consola de AWS



Figura 12: Reglas de entrada del Security Group (puertos 22 y 80)

5.7. Seguridad: GitHub Secrets

Ningún dato sensible se escribió en el código. En el repositorio se configuraron cinco secretos (Figura 13): `DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`, `EC2_HOST` (IP pública de la instancia), `EC2_USER` y `EC2_SSH_KEY` (contenido del archivo `.pem`). GitHub los cifra, los inyecta solo durante la ejecución y los oculta (***) en los logs. Como medida adicional, `.gitignore` y `.dockerignore` excluyen archivos `.env`, `*.pem` y `*.key`.

Figura 13: Secretos configurados en el repositorio (GitHub nunca muestra sus valores)

5.8. Despliegue continuo en EC2

El job `deploy` se conecta a la instancia por SSH con `appleboy/ssh-action` y ejecuta el siguiente proceso:

1. Descarga la imagen `:latest` recién publicada en Docker Hub.
2. Levanta la versión nueva en un contenedor temporal (puerto 8081) y consulta `/api/health` hasta 20 veces. Si no responde, muestra sus logs, lo elimina y marca el pipeline como fallido **sin tocar** la versión en producción.
3. Si está sana, elimina el contenedor temporal, detiene el contenedor antiguo (`api`) y levanta la nueva versión mapeada al puerto 80 con `--restart unless-stopped`.
4. Verifica que `/api/health` responda con el hash del commit del *push*, lo que comprueba que realmente se está sirviendo la versión nueva, y limpia imágenes viejas.

Como la imagen ya está descargada y validada antes de detener la versión anterior, el cambio de contenedor tarda alrededor de un segundo, reduciendo el tiempo de inactividad al mínimo. En las pruebas realizadas, el job completo de despliegue tardó unos 10 segundos. La Figura 14 muestra el log del despliegue, la Figura 15 el contenedor corriendo en la EC2 y la Figura 16 la API respondiendo en la IP pública.

```
GitHub Actions — Despliegue en AWS EC2 › Desplegar nueva versión por SSH

Drone SSH version 1.8.2
=====
>> Descargando ***/sa-cicd-api:latest desde Docker Hub
latest: Pulling from ***/sa-cicd-api
44136fa355b3: Already exists
d62eef790cef: Download complete
Digest: sha256:e1cfbc4e0d8d483a9fdb9194f96ae8d65d65122b1daba7301c9cfe6b13ef23bb
Status: Downloaded newer image for ***/sa-cicd-api:latest
docker.io/***/sa-cicd-api:latest
5e544dd07379065880d4ec322310e62eea5e18e6bc105b38d2d6b97bedec173
api-candidate
>> Reemplazando contenedor en el puerto 80
api
0c9bceccde6b7332286bf0291fbabbe8b0b4603e3e312857703c7b4737c4e017
>> Despliegue OK, commit 47737e5dfc45314500b24fcaedbc98a6c3a145a4 en línea
Deleted Images:
untagged: sha256:2a7873c2cbd4119f75e03839206d1eebe90ce649feef892e481d8cf39ca5683c
deleted: sha256:2a7873c2cbd4119f75e03839206d1eebe90ce649feef892e481d8cf39ca5683c
deleted: sha256:4409466d710e29c69e805ab4fd2d07b9a28ba87806533138c97298556387485d
deleted: sha256:dc21770cd8fde18535ae009c34e26a2a8d4583c004ec553518c8b8e087b17e33
Total reclaimed space: 37.49kB
=====
✅ Successfully executed commands to all hosts.
```

Figura 14: Log del job deploy en GitHub Actions

```
ubuntu@ip-172-31 (EC2) — docker ps

$ ssh -i webapp-key.pem ubuntu@18.223.115.56
$ docker ps
CONTAINER ID   IMAGE                                STATUS      PORTS
NAMES
0c9bceccde6b   kusco13/sa-cicd-api:latest          Up 4 minutes (healthy)   0.0.0.0:80->8000/tcp, [::]:80->8000/tcp
api

$ docker images
REPOSITORY    TAG       IMAGE ID       SIZE
kusco13/sa-cicd-api   latest   e1cfbc4e0d8d   205MB
```

Figura 15: Contenedor api en ejecución en la EC2 (docker ps)

```
Impresión con sangría ☐

{"statusCode":200,"data":{"status":"ok","version":"1.0.0","message":"Demo en vivo: versión 2 desplegada automáticamente - IDGS16","commit":"1a6343172225563af2b0f0b1db6db7c13c797616"}}
```

Figura 16: API respondiendo en el navegador en <http://18.223.115.56/api/health>

5.9. Demostración práctica

Para la demostración en vivo se modificó una sola línea del archivo `config.py`, la constante `MESSAGE`, y se hizo `git commit` y `git push` a `main`. Antes del cambio, `GET /api/health` mostraba el mensaje “Hola desde el pipeline CI/CD - IDGS16” y el commit `47737e5` (Figura 17). El `push` disparó la ejecución #6 del pipeline (Figura 8), que en 59 segundos ejecutó las pruebas, publicó la etiqueta `1a63431...` en Docker Hub (Figura 10) y actualizó el servidor. Al volver a consultar la URL se obtuvo el mensaje nuevo, “Demo en vivo: versión 2

desplegada automáticamente - IDGS16", junto con el commit 1a63431 (Figura 18), sin haber entrado manualmente al servidor.

```
PowerShell — Antes del cambio: estado actual y git push

$ curl http://18.223.115.56/api/health
{
  "statusCode": 200,
  "data": {
    "status": "ok",
    "version": "1.0.0",
    "message": "Hola desde el pipeline CI/CD - IDGS16",
    "commit": "47737e5dfc45314500b24fcaedbc98a6c3a145a4"
  }
}

$ git diff config.py
@@ -4,4 +4,4 @@ Para la demostración en vivo basta con cambiar MESSAGE, hacer commit y git push
el pipeline corre las pruebas, publica la imagen y actualiza la EC2.
"""
VERSION = "1.0.0"
-MESSAGE = "Hola desde el pipeline CI/CD - IDGS16"
+MESSAGE = "Demo en vivo: versión 2 desplegada automáticamente - IDGS16"
$ git commit -am "Demo: cambia mensaje de la API"
[main 1a63431] Demo: cambia mensaje de la API
1 file changed, 1 insertion(+), 1 deletion(-)
$ git push
To https://github.com/2023371031-del/sa-cicd-api.git
47737e5..1a63431  main -> main
```

Figura 17: /api/health antes del cambio, y el git commit / git push de la demo

```
PowerShell — Después del pipeline (≈50 s)

$ curl http://18.223.115.56/api/health
{
  "statusCode": 200,
  "data": {
    "status": "ok",
    "version": "1.0.0",
    "message": "Demo en vivo: versión 2 desplegada automáticamente - IDGS16",
    "commit": "1a6343172225563af2b0f0b1db6db7c13c797616"
  }
}
```

Figura 18: /api/health después del git push: mensaje y commit nuevos

6. Conclusiones

El proyecto permitió construir de principio a fin un flujo de trabajo CI/CD como el que se usa en la industria. Se desarrolló una API REST con 6 endpoints, organizada en capas de rutas, servicios y datos, y respaldada por 31 pruebas automatizadas con una cobertura del 99.48 %. Más allá del número, las pruebas resultaron ser la pieza que da confianza a todo el pipeline: como el umbral de cobertura y el resultado de las pruebas son condiciones obligatorias para continuar, ningún cambio que rompa la API puede llegar a Docker Hub

ni al servidor. Separar el código en rutas, servicios y datos también demostró su valor: las reglas de negocio (validaciones, email único) viven en un solo lugar, lo que facilita probarlas a fondo y agregar nuevos recursos en el futuro sin tocar la lógica existente.

La contenedorización con Docker resolvió el problema de las diferencias entre entornos: la misma imagen que se probó localmente es la que se publica y se ejecuta en EC2, identificada con el hash de su commit, lo que facilita saber qué versión está en producción y regresar a una anterior si es necesario. GitHub Actions unió todas las piezas en un solo proceso reproducible y visible para todo el equipo, y GitHub Secrets permitió automatizar el acceso a Docker Hub y a AWS sin exponer contraseñas, tokens, IPs ni llaves en un repositorio público, que es uno de los errores de seguridad más comunes en proyectos reales.

Finalmente, la demostración práctica evidenció el beneficio principal de DevOps: un cambio de una línea pasó de la computadora del desarrollador a producción en pocos minutos, sin intervención manual y con verificaciones automáticas de salud antes y después del despliegue. Como trabajo futuro, la solución podría incorporar una base de datos persistente (por ejemplo, Amazon RDS), un proxy inverso con despliegue *blue-green* para eliminar por completo el segundo de inactividad durante el cambio de contenedor, y HTTPS mediante un certificado TLS.

Fuentes de información

- [1] Docker Inc. (2026). *Building best practices*. Docker Documentation. <https://docs.docker.com/build/building/best-practices/>
- [2] Pallets Projects. (2026). *Flask documentation (3.1.x)*. <https://flask.palletsprojects.com/>
- [3] Fowler, M. (2006). *Continuous integration*. martinowler.com. <https://martinfowler.com/articles/continuousIntegration.html>
- [4] GitHub. (2026). *GitHub Actions documentation*. GitHub Docs. <https://docs.github.com/en/actions>
- [5] Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.
- [6] Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations*. IT Revolution Press.

- [7] Amazon Web Services. (2026). *Amazon EC2 user guide*. AWS Documentation. <https://docs.aws.amazon.com/ec2/>